SYSTEM DESIGN

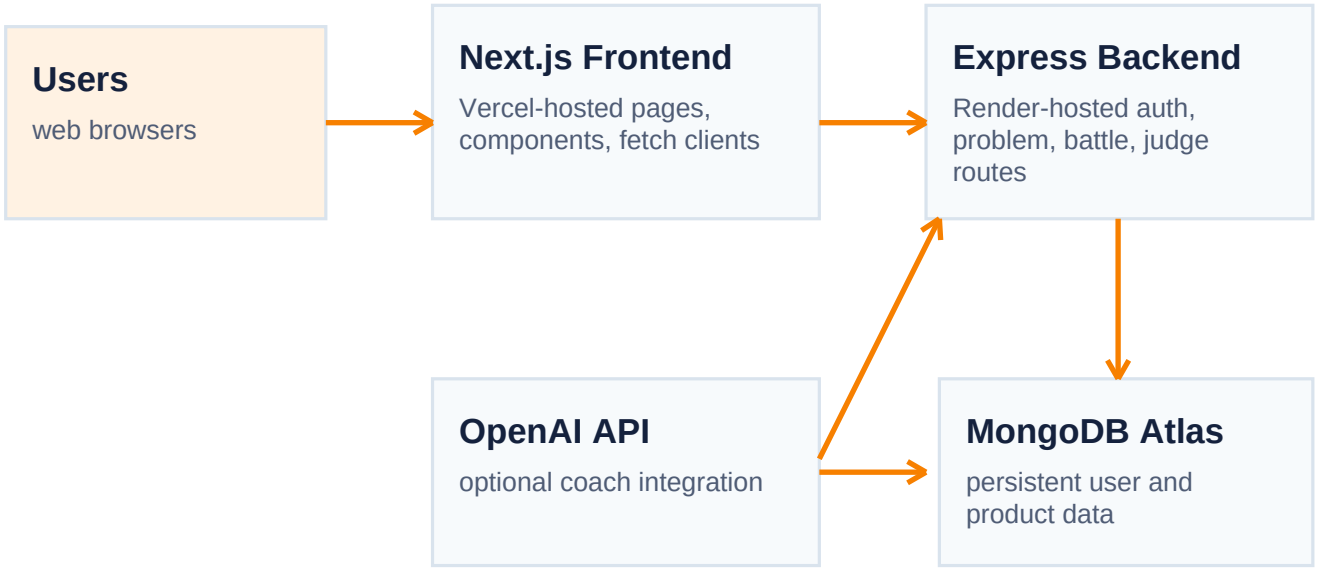
SmartCode System Design Document

Production-oriented design for the current SmartCode architecture, including present constraints and recommended next improvements.

System objective

SmartCode is a coding-practice platform where users can sign up, log in, browse coding problems, run and submit solutions, track progress, compete in battle mode, and optionally receive AI coaching. The current system is optimized for fast product iteration and launch speed.

Current architecture



This separation is good for early production because frontend and backend deploy independently, while Atlas provides managed persistence.

Major functional modules

Module	Responsibility	Current note
Auth	Signup, login, profile updates, password changes, presence	Needs stronger password security
Problems	List, details, favorites, run, submit, study plans	Most important user-value path
Judge	Evaluate JavaScript, TypeScript, and Java solutions	High reliability requirement
Battle	Queue, match, score, and persist contest outcomes	Monitor carefully for edge cases
Coach	Fallback hints or OpenAI-based hints	Should not block core product if unavailable

Data model overview

Important product state

Progress, rankings, favorites, recent activity, and problem metadata all live in MongoDB Atlas.

Recovery principle

Back up data before migrations or dangerous bulk edits, and keep GitHub as code truth.

Scalability and reliability

- Current strengths: simple service boundaries, managed infrastructure, and fast independent deploys.
- Current constraints: Render cold starts, no automated regression tests, and limited structured observability.
- Near-term improvements: add request validation, health checks, route smoke tests, and structured logs with request identifiers.
- Scale-up path: move to an always-on backend tier, add rate limiting, and separate the judging workload if submission traffic grows significantly.

Security priorities

- Hash passwords with bcrypt before broader public growth.
- Rotate exposed MongoDB credentials and keep secrets only in hosting dashboards.
- Restrict CORS to exact trusted frontend domains.
- Consider token-based or session-based auth hardening over time.

Observability and debugging strategy

Area	Current tool	Recommended next step
Frontend failures	Browser console and Vercel deploy logs	Add client-side error reporting
Backend failures	Render logs	Add structured route-level logging
Database health	MongoDB Atlas metrics	Track slow queries and usage growth weekly
AI dependency	Fallback behavior	Log coach mode to know when OpenAI is or is not active